

Mastering Advanced Java & DSA - Jan '26



Day – 3 [Object-Oriented Programming in Java]

- Introduction to OOP in Java
- Creating Classes and objects
- Inheritance
- Polymorphism
- Encapsulation
- Abstraction



Introduction to OOP in Java

OOP Principles

- Before Object-Oriented Programming (OOPs), most programs used a procedural approach, where the focus was on writing step-by-step functions. This made it harder to manage and reuse code in large applications.
- To overcome these limitations, Object-Oriented Programming was introduced. Java is built around OOPs, which helps in organizing code using classes and objects.

Key Features of OOP in Java

- Structures code into logical units (classes and objects)
- Keeps related data and methods together (encapsulation)
- Makes code modular, reusable and scalable
- Prevents unauthorized access to data
- Follows the DRY (Don't Repeat Yourself) principle

Characteristics of an OOP(Object Oriented Programming)



1. Class

A Class is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. Using classes, you can create multiple objects with the same behavior instead of writing their code multiple times. In general, class declarations can include these components in order:

- **Modifiers:** A class can be public or have default access
- **Class name:** The class name should begin with the initial letter capitalized by convention.
- **Body:** The class body is surrounded by braces, { }.

Example:

```
class Car {  
    String brand;  
    int year;  
  
    void displayDetails() {  
        System.out.println("Car Brand: " + brand);  
        System.out.println("Car Year: " + year);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.brand = "Honda";  
        myCar.year = 2022;  
        myCar.displayDetails();  
    }  
}
```

Explanation:

`class Car {` → Defines a class `Car`.

`String brand;` → Variable for car brand.

`int year;` → Variable for car year.

`void displayDetails() {` → Method to show details.

`System.out.println("Car Brand: " + brand);` → Prints brand.

`System.out.println("Car Year: " + year);` → Prints year.

`}` → Ends method.

`}` → Ends class `Car`.

`public class Main {` → Defines `Main` class.

`public static void main(String[] args) {` → Program starts here.

`Car myCar = new Car();` → Creates object `myCar`.

`myCar.brand = "Honda";` → Sets brand.

`myCar.year = 2022;` → Sets year.

`myCar.displayDetails();` → Calls method, prints details.

`}` → Ends `main`.

`}` → Ends `Main` class.

2. Object

- An object is an instance of a class that represents a real-world entity.
- It contains state (fields/variables) and behavior (methods).

Example

```
class Student {  
    String name;  
    int age;  
  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.name = "Abinaya";  
        s1.age = 21;  
        s1.display();  
    }  
}
```

Explanation:

- Student is the class (blueprint).
- s1 is the object created from the class.
- Using the object, we assign values (name, age) and call the method display().

3. Abstraction

- Abstraction hides implementation details and shows only the essential features to the user.
- It is achieved using abstract classes and interfaces in Java

Example

```
abstract class Animal {  
    abstract void sound();  
  
    void sleep() {  
        System.out.println("Sleeping...");  
    }  
}
```

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Barks");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Dog();  
        a.sound();  
        a.sleep();  
    }  
}
```

Explanation:

abstract class Animal → Contains both abstract and normal methods.

abstract void sound() → Must be implemented by child classes.

class Dog extends Animal → Provides implementation of sound().

In main(), we create an Animal reference but store a Dog object.

Calls sound() → prints Barks.

Calls sleep() → prints Sleeping....

4. Encapsulation

- Encapsulation is the process of wrapping data (variables) and methods into a single unit (class).
- It is implemented by declaring variables as private and providing getters/setters for controlled access.

Example:

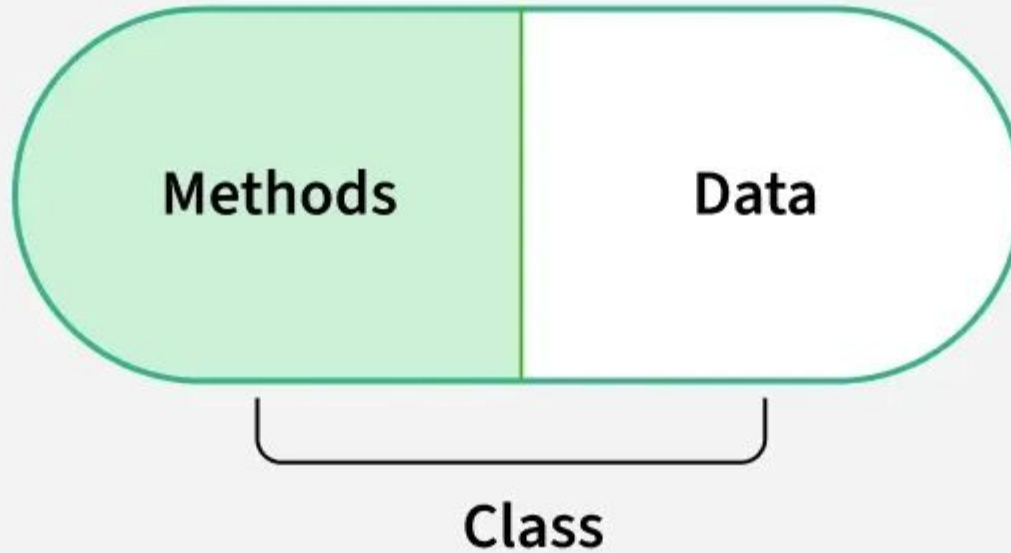
```
class Student {  
    private String name;  
    private int age;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
    public int getAge() {  
        return age;  
    }  
}
```

```
    public void setAge(int age) {  
        this.age = age;  
    }  
}  
public class Main {  
    public static void main(String[]  
args) {  
        Student s = new Student();  
        s.setName("Abinaya");  
        s.setAge(21);  
        System.out.println("Name: " +  
s.getName());  
        System.out.println("Age: " +  
s.getAge());  
    }  
}
```

Explanation:

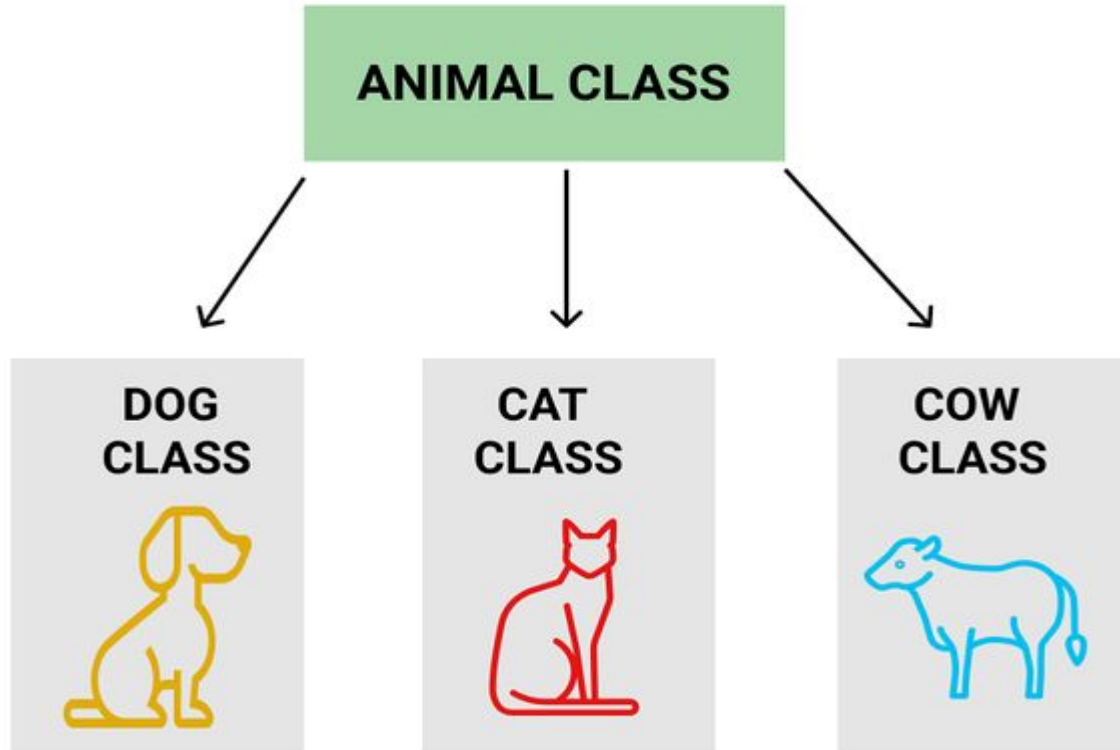
- Variables name and age are private → can't be accessed directly.
- Access is controlled using getters and setters.
- In main(), we use setters to assign values and getters to retrieve them.

Encapsulation



5. Inheritance

- Inheritance allows a class (child/subclass) to acquire the properties and methods of another class (parent/superclass).
- It promotes code reusability and supports hierarchical relationships.



- Let us discuss some frequently used important terminologies:
- Superclass: The class whose features are inherited is known as superclass (also known as base or parent class).
- Subclass: The class that inherits the other class is known as subclass (also known as derived or extended or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- Reusability: Inheritance supports the concept of "reusability", i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Example

```
class Person {
    String name;
    int age;
    void displayPerson() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}
class Student extends Person {
    String course;
    void displayStudent() {
        System.out.println("Course: " + course);
    }
}
```

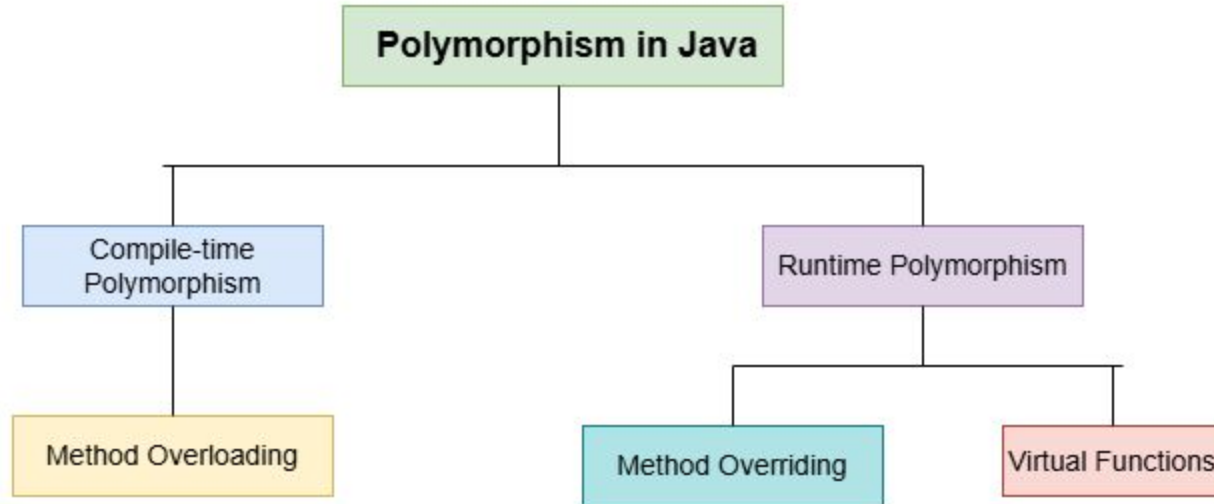
```
public class Main {
    public static void main(String[]
args) {
        Student s = new Student();
        s.name = "Abinaya";
        s.age = 21;
        s.course = "Computer
Science";
        s.displayPerson();
        s.displayStudent();
    }
}
```

Explanation:

- Person is the parent class.
- Student is the child class that inherits properties (name, age) and methods of Person.
- Object s of Student can access both parent and child members.

6. Polymorphism

The word polymorphism means having many forms, and it comes from the Greek words poly (many) and morph (forms), this means one entity can take many forms. In Java, polymorphism allows the same method or object to behave differently based on the context, specially on the project's actual runtime class.



Types of Polymorphism

Polymorphism in Java is mainly of 2 types as mentioned below:

1. Method Overloading
2. Method Overriding

Method Overloading and Method Overriding:

1. **Method Overloading:** Also, known as compile-time polymorphism, is the concept of Polymorphism where more than one method share the same name with different signature(Parameters) in a class. The return type of these methods can or cannot be same.

2. **Method Overriding:** Also, known as run-time polymorphism, is the concept of Polymorphism where method in the child class has the same name, return-type and parameters as in parent class. The child class provides the implementation in the method already written.

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Dog();  
        Animal a2 = new Cat();  
        a1.sound();  
        a2.sound();  
    }  
}
```

Explanation:

- Polymorphism = One name, many forms.
- sound() is the same method but behaves differently depending on the object (Dog or Cat).
- This is runtime polymorphism (method overriding).

Recommendations

1. Practice More Problems → Solve DSA coding challenges daily on platform CodeKatta.
2. Hands-on with Java Features → Implement custom collections, thread-safe programs, and practice design patterns.
3. Mini Projects → Build a Student Management System, Online Library, or Task Scheduler to apply Java + DSA together.
4. System Design Practice → Try designing Parking Lot, E-commerce Cart, or Chat Application using OOP + SOLID principles.
5. Deep Dive into DSA → Focus on problem patterns: Sliding Window, Binary Search on Answer, Graph Traversals, DP on Trees.
6. Competitive Programming → Set daily/weekly goals to solve problems under time limits to mimic interview pressure.

1. Code with Functional Style → Use Java 8+ features (Streams, Lambdas, Optional) in all projects for clean, modern coding.
2. Revise Complexity Analysis → Always analyze Big-O for your code; optimize where possible with better data structures.
3. Mock Assessments → Attempt coding tests (similar to HackerRank/CodeSignal) after each module to track progress.
4. Prepare for Interviews → Practice common interview patterns: Linked List cycle detection, DP Knapsack, Graph shortest path, and LLD problems.

Thank You!